

×



Screen Object Pattern with MobileWright

A line-by-line technical guide to the framework

Reference document for onboarding DXC software testers onto the MobileWright mobile automation framework.

Running example: automating a YouTube scenario on Android

Written by

ELMATTAD Zouhair — Software Tester

English version — August 11, 2026

Internal Document — Confidential. Property of DXC Technology. For internal use only; any distribution outside the company is prohibited without authorization.

© 2026 DXC Technology — All rights reserved.

Contents

1 Before you start 3

2 What the pattern must guarantee 3

3 A bird's-eye view before the details 4

4 Identifying the screens and defining the repository structure 4

5 ScreenBase: factoring out what every screen has in common 6

5.1 The import: fetching only type contracts . . . 6

5.2 The protected field: visibility and immutability . . . 6

5.3 The constructor: receiving the connection rather than building it . . . 7

5.4 Inheritance: what every screen receives for free . . . 7

6 Locators: isolating the question "Where is the element?" 7

6.1 Declaring the field: why `public` . . . 8

6.2 The constructor: a shorthand syntax specific to TypeScript . . . 8

6.3 A deliberate choice: eager locator initialization . . . 8

6.4 The same principle, repeated for YouTube . . . 8

7 Screen classes: turning technical operations into business actions 8

7.1 Three imports, three purposes . . . 9

7.2 Inheritance and the constructor . . . 9

7.3 The private locator: a hidden implementation detail . . . 10

7.4 Methods named by intent . . . 10

7.5 YouTubeScreen: the same principle, one business nuance . . . 10

8 ScreenManager: the conductor, explained line by line 11

8.1 The imports: three lines, three different natures . . . 11

8.2 Declaring the class and exporting it . . . 11

8.3 The class fields: visibility explained through a real contrast . . . 11

8.4 The constructor: the moment three objects come to life . . . 12

8.5 The accessor methods: why return rather than recreate . . . 12

9 Fixtures: wiring the framework into the test runner 13

9.1 The `Fixtures` type: describing a resource before providing it . . . 13

9.2 Extending the base test system . . . 13

9.3 What the provided function actually does . . . 14

9.4 The precise role of `use(...)` 14

10 The scenario: concrete proof of the benefit **14**

10.1 What this file never does 15

10.2 What this file tells 15

10.3 The proof of maintainability 15

11 In summary **15**

1 Before you start

This document explains, in depth, how the mobile automation framework used in this project is built and, above all, **why** each design decision was made. To keep the explanations concrete, we will rely on a real scenario: opening YouTube from the home screen of an Android phone, searching for a video, then interacting with it.

If you are new to programming, keep one essential idea in mind before reading further: a mobile application is made up of a succession of **screens** (home screen, YouTube, settings, etc.), and each screen contains **elements** the user can interact with, such as a button, a search field, or a video. Automating a test simply means reproducing, through code, the actions a user would perform manually on these different screens.

The **Screen Object Pattern** is a way of organizing code that maps each screen to a dedicated file and each interaction to a clearly defined method. This structure makes the framework more readable, easier to maintain, and simpler to evolve.

The rest of this document starts from the most fundamental concepts and progressively builds the framework, file by file. Particular attention will be paid to the central file `screenManager.ts`, which is the heart of the architecture. We will explain in detail the main technical decisions, including field visibility, the role of the constructor, export mechanisms, and object creation and management.

WHO IS THIS DOCUMENT FOR?

This document is primarily aimed at software testers who are discovering mobile automation and the TypeScript language. No prior knowledge of object-oriented programming is required: every notion (class, object, inheritance, encapsulation...) is introduced exactly where it appears in the code, along with its reason for being. Very general JavaScript or TypeScript basics (variables, functions, imports) will make the reading easier, without being mandatory.

CLASS, OBJECT, INSTANCE: THREE WORDS FOR ONE IDEA

Before going further, a clarification of vocabulary avoids a lot of confusion. A **class** is a blueprint: it describes the shape of something (its fields, its methods), but it does not concretely exist until someone uses it with the `new` keyword. An **object** is the thing actually built from that blueprint. Every object created this way is called an **instance** of its class. In this document, whenever we write that a constructor "creates an object", it always means: an instance of that class has just been built in memory, ready to be used.

2 What the pattern must guarantee

The goal of an automation framework is not just to make a test pass once. It must, above all, be designed to withstand the application's evolution over time. Indeed, a mobile application keeps evolving: new screens appear, buttons change position, identifiers are modified, and new features are added with every release. The framework must therefore be able to adapt to these changes while limiting the modifications required.

To reach this goal, three fundamental qualities are pursued.

Readability — A test scenario should read almost like a sentence in plain language. For example, the instruction `screenManager.onHomeScreen().clickOnHomeButton()` makes it immediately clear that the test taps the *Home* button, without needing to know the technical details of that interaction.

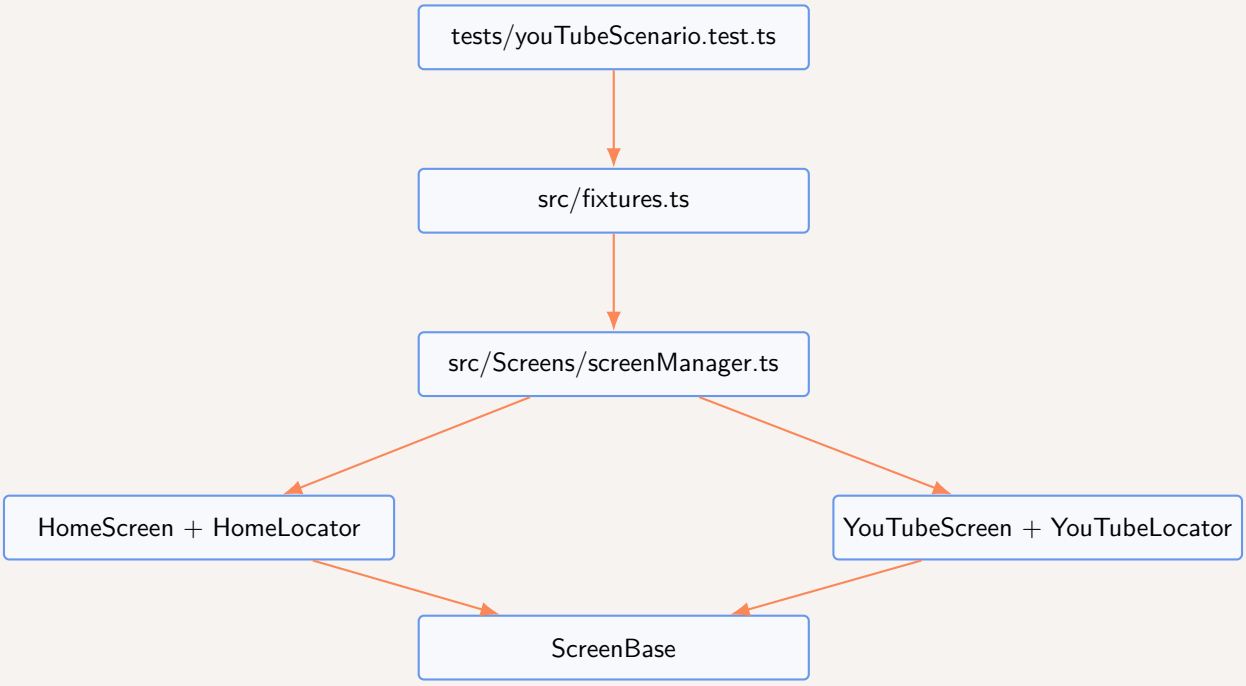
Maintainability — When an element of the application changes, for example a button's identifier, the fix should be made in exactly one place: the file containing the corresponding *locator*. The test scenarios themselves remain unchanged, which drastically reduces maintenance effort.

Scalability — The framework must be able to grow easily. Adding a new screen or a new feature should not require modifying the screens that already exist. Each screen is an independent module that can be plugged into the framework without impacting the rest of the architecture.

To guarantee these three properties, the framework applies the principle of **separation of concerns**. Its organization directly mirrors the structure of the application under test: each mobile screen has its own folder, and each technical responsibility is isolated in a dedicated file. *Locators* are responsible for identifying elements, *screens* group the actions available on those elements, while the *Screen Manager* orchestrates the interactions between the different screens. This separation makes the framework clearer, more robust, and far easier to maintain over the long term.

3 A bird's-eye view before the details

Before examining the code in detail, it helps to have a global view of the framework's architecture. The diagram below shows the full chain of components, from the test scenario down to the MobileWright automation engine. This overview will make it easier to understand the role of each element before studying its implementation.



This diagram highlights the different layers that make up the framework, and how they relate to each other. In the following sections, each of these layers will be studied progressively, starting from the lowest level – the shared technical building blocks – up to the highest level, where readable, plain-language test scenarios are defined.

Why? shouldn't you try to memorize everything here? This diagram is a mental map, not an exam. Every file name and every arrow will be picked up again and explained in detail, one by one, in the chapters that follow.

4 Identifying the screens and defining the repository structure

Before writing a single line of code, the first step is to identify the **screens** (or *testwares*) that the framework will need to represent. The framework is designed to reproduce the interactions a user

performs on the application, so every screen touched by the test scenarios must be modeled by a dedicated software component.

In this project, the automation scenario uses two main screens: the Android phone's home screen (**Home**) and the **YouTube** application. These two screens are the first components of the framework.

Once these screens are identified, their organization directly shapes the structure of the repository, which is deliberately split into two parts with clearly distinct responsibilities:

- `src/` contains the **automation framework**, i.e. the set of reusable components used to interact with the application.
- `tests/` contains the **test scenarios**, which use the framework to reproduce complete functional journeys.

Inside the `src/Screens/` folder, each screen is represented by its own folder, whose name matches the screen it models.

```
src/
  Screens/
    screenBase.ts
    screenManager.ts
    Home/
      home.locator.ts
      home.screen.ts
    YouTube/
      youtube.locator.ts
      youtube.screen.ts
  fixtures.ts
tests/
  youTubeScenario.test.ts
```

This organization was not chosen at random. One could, for example, group all screens into a single file. However, such an approach would quickly make the project difficult to maintain.

Why? create a separate folder for each screen?

Each screen has its own lifecycle and evolves independently of the others. For example, a YouTube app update can change its interface without any change being required on the phone's home screen. By grouping every file related to a single screen inside a dedicated folder, changes stay localized. When a screen evolves, only one folder is affected, without impacting the rest of the framework. This organization considerably improves maintainability, readability, and scalability.

Inside each screen folder, exactly two files — and only two — are created.

The first, named `*.locator.ts`, describes **the elements present on the screen**, i.e. the interface components the framework must be able to identify (buttons, search fields, videos, etc.).

The second, named `*.screen.ts`, describes **the actions available on that screen**. It groups all possible interactions with the elements defined in the *locator* file, such as tapping a button, entering text, or selecting a video.

This separation between **interface elements** (*locators*) and **business actions** (*screens*) is one of the framework's foundational principles. It simplifies maintenance, encourages code reuse, and will serve as the common thread running through every section that follows.

5 ScreenBase: factoring out what every screen has in common

Before building a specific screen, it is important to identify the capabilities that every screen shares. Certain operations, for instance, do not depend on any particular screen: tapping an element, entering text into a field, scrolling the screen, or performing a swipe gesture in a given direction.

If these operations were implemented directly inside each screen class, the same code would be duplicated across `HomeScreen`, `YouTubeScreen`, and every future screen in the project. This duplication would make the framework harder to maintain and increase the risk of bugs. To avoid this, all shared capabilities are grouped inside a base class, `ScreenBase`, located in the file `src/Screens/screenBase.ts`. The other screen classes will then inherit from this class in order to automatically reuse these capabilities. We break down every decision in this file below, in the order it is written.

```
import type { Locator, Screen } from '@mobilewright/core';

export class ScreenBase {
  protected readonly screen: Screen;

  constructor(screen: Screen) {
    this.screen = screen;
  }

  async clickOnApplicationIcon(locator: Locator): Promise<void> {
    await locator.tap();
  }

  async swipeUpFromScreenPoint(startX: number, startY: number, distance: number,
      duration?: number): Promise<void> {
    await this.screen.swipe('up', { startX, startY, distance, duration });
  }

  // ... other shared gestures:
  // swipeDown, fillTextField, tap,
  // scroll, swipeLeft, swipeRight...
}
```

5.1 The import: fetching only type contracts

```
import type { Locator, Screen } from '@mobilewright/core';
```

TYPE IMPORT

This line imports only two **types**: `Screen` and `Locator`. Unlike a regular import, it adds no code to the final program: it is used exclusively by the TypeScript compiler, to verify that the manipulated objects really do have the expected methods, such as `tap()` or `swipe()`. Once this verification is done, the statement disappears when the JavaScript code is generated.

Why? type the `locator` parameter with `Locator` rather than with `any`? `any` would disable every compiler check on that variable: any method could be called on it, even if it does not exist, without a single warning before runtime. That would be inconsistent with the rest of this chapter, which precisely values strong typing. By importing the real `Locator` type provided by `@mobilewright/core`, the compiler verifies that `locator.tap()` actually exists, and auto-completes it in the editor.

5.2 The protected field: visibility and immutability

```
protected readonly screen: Screen
```

VISIBILITY AND IMMUTABILITY

Why? choose `protected` rather than `private` or `public`? The `protected` keyword makes the field accessible only inside the `ScreenBase` class and the classes that inherit from it, such as `HomeScreen` or `YouTubeScreen`; it stays inaccessible from the rest of the application. It is a deliberate compromise between `private`, which would also block access from child classes, and `public`, which would make the field accessible from anywhere in the project. We will revisit this comparison more fully in the chapter dedicated to `screenManager.ts`, where all three visibility levels coexist inside the same class.

The `readonly` keyword, in turn, indicates that the reference held in `screen` can only be assigned once: once the object is created, it can never be replaced. This choice guarantees that the connection to the mobile device stays the same throughout the entire test run.

5.3 The constructor: receiving the connection rather than building it

```
constructor(screen: Screen) {
  this.screen = screen;
}
```

DEPENDENCY INJECTION

Why? receive `screen` as a parameter rather than creating it with `new`? This approach follows the **dependency injection** principle: the class does not concern itself with how the connection to the device is established, it simply receives an already-initialized connection. This separation of concerns improves the framework's flexibility and makes it easier to write unit tests, in which a real device can be replaced with a simulated object (a *mock*) that reproduces the same behavior.

5.4 Inheritance: what every screen receives for free

INHERITANCE

The classes representing the different screens, such as `HomeScreen` or `YouTubeScreen`, inherit from `ScreenBase` through the `extends` keyword. This mechanism, called **inheritance**, lets every screen automatically reuse all the shared capabilities defined in the base class, such as `clickOnApplicationIcon()` or `swipeUpFromScreenPoint()`, without re-implementing them. Every screen class can therefore focus solely on the interactions specific to it, which reduces code duplication and improves both readability and maintainability.

6 Locators: isolating the question "Where is the element?"

Once the shared capabilities are defined in `ScreenBase`, the next step is to let the framework locate the elements present on each screen. Before you can tap a button, type text, or select a video, you first need to know where the relevant element sits in the interface. That is precisely the role of the files `home.locator.ts` and `youtube.locator.ts`: describing the user-interface elements, without performing any action. We break down the file `home.locator.ts` below, line by line.

```
import type { Screen } from '@mobilewright/core';

export class HomeLocator {
  public readonly youTubeApplicationIconLocator;

  constructor(private readonly screen: Screen) {
    this.youTubeApplicationIconLocator =
      this.screen.getByRole('text', { name: 'YouTube' });
```

```
  }
}
```

6.1 Declaring the field: why `public`

```
public readonly youTubeApplicationIconLocator
```

VISIBILITY

Why? make this field `public`, when the `screen` field on `ScreenBase` is `protected`? Unlike that internal resource, which must stay protected, a *locator* is designed to be used by screen classes, such as `HomeScreen`. Its only role is to provide an access point to an interface element: there is therefore no reason to hide it behind `private`. Making it public simplifies its use without compromising encapsulation.

6.2 The constructor: a shorthand syntax specific to TypeScript

```
constructor(private readonly screen: Screen) {
    this.youTubeApplicationIconLocator =
      this.screen.getByRole('text', { name: 'YouTube' });
```

PARAMETER PROPERTIES

This syntax introduces a feature called **parameter properties**. Instead of declaring a field and then initializing it by hand inside the constructor's body, as `ScreenBase` does, it performs both steps in a single statement: the compiler automatically generates a field named `screen`, applies the `private` and `readonly` modifiers to it, and initializes it with the value received as a parameter. This syntax is strictly equivalent to the longer form, but more concise and widely used across TypeScript projects.

6.3 A deliberate choice: eager locator initialization

EAGER LOCATOR

Why? compute `this.screen.getByRole(...)` once, in the constructor, rather than on every use? This strategy fits this document's scenario perfectly, because the screens' structure stays stable between the creation of `ScreenManager` and the execution of the various actions. It does, however, reach its limits if an application generates or dynamically changes its interface: in that case, it would be preferable to recompute the *locator* on every use, for example through a method or a `get` accessor, so the framework always works against the most recent version of the interface.

6.4 The same principle, repeated for YouTube

The file `youtube.locator.ts` applies exactly the same principle to five elements: the search icon, the search field, the video, the button that dismisses the ad, and the video's progress slider. No new design rule is introduced there; this repetition is deliberate: applying the same structure consistently to every screen makes the framework more homogeneous, more predictable, and therefore easier to understand, maintain, and evolve.

7 Screen classes: turning technical operations into business actions

The `*.screen.ts` files form the layer that connects `ScreenBase`'s shared capabilities to the elements described in the *locator* files. Their role is no longer to locate an element or run an isolated technical

gesture, but to combine these building blocks into business actions expressed in language close to the user's own. In other words, a *Screen* class answers the question: *what does the user want to do on this screen?*

The file `home.screen.ts` illustrates this principle perfectly.

```
import { ScreenBase } from '../screenBase';
import type { Screen } from '@mobilewright/core';
import { HomeLocator } from './home.locator';

export class HomeScreen extends ScreenBase {
  private readonly homeLocator: HomeLocator;

  constructor(screen: Screen) {
    super(screen);
    this.homeLocator = new HomeLocator(screen);
  }

  async clickOnHomeButton(): Promise<void> {
    await this.screen.pressButton('HOME');
  }

  async clickOnYouTubeApplicationIcon(): Promise<void> {
    await this.clickOnApplicationIcon(
      this.homeLocator.youTubeApplicationIconLocator
    );
  }

  async middleSwipeUp(): Promise<void> {
    await this.swipeUpFromScreenPoint(238, 551, 1000);
  }

  async bottomSwipeUp(): Promise<void> {
    await this.swipeUpFromScreenPoint(224, 793, 1000);
  }
}
```

7.1 Three imports, three purposes

VALUE IMPORT OR TYPE IMPORT

The three `import` statements at the top of the file do not serve the same purpose. `import { ScreenBase } from '../screenBase'` imports an actual class, which `HomeScreen` needs in order to inherit its methods through `extends`. `import type { Screen }` merely restates, as a reminder, the type check already explained in the previous chapter: this import disappears entirely at compile time. Finally, `import { HomeLocator } from './home.locator'` also imports an actual class, since an instance of it is created a little further down with `new HomeLocator(screen)`.

7.2 Inheritance and the constructor

INHERITANCE

The line `extends ScreenBase` implements the **inheritance** mechanism: `HomeScreen` automatically gains every shared capability defined in `ScreenBase`, such as `clickOnApplicationIcon()` or `swipeUpFromScreenPoint()`, without having to rewrite them. The constructor must start with the call `super(screen)`, required whenever a class inherits from another class that itself has a constructor: this statement first runs `ScreenBase`'s constructor, which initializes the `screen` field, before the child class

can use its own `this` object to continue its initialization.

7.3 The private locator: a hidden implementation detail

ENCAPSULATION

Why? declare `private readonly homeLocator: HomeLocator`, when the fields it holds (such as `youTubeApplicationIconLocator`) are `public`? Unlike the *locators* it contains, the `homeLocator` object is an implementation detail specific to `HomeScreen`, one that nothing outside needs to know about. An external class must never manipulate `HomeScreen`'s *locators* directly, only call the services it exposes, such as `clickOnYouTubeApplicationIcon()`. This encapsulation limits dependencies between classes and improves the framework's maintainability.

7.4 Methods named by intent

`HomeScreen`'s methods are deliberately named after a functional logic rather than a technical one: `clickOnYouTubeApplicationIcon()`, `middleSwipeUp()`, or `bottomSwipeUp()` describe the action performed by the user, without ever exposing the details of their implementation. It is precisely this vocabulary that will later allow test scenarios to read almost like plain-language sentences.

7.5 YouTubeScreen: the same principle, one business nuance

The file `youtube.screen.ts` applies exactly the same architectural principles: inheriting from `ScreenBase`, calling `super()`, a private locator, methods named by intent. One nuance deserves to be pointed out: `clickOnSkipAdButton()` first checks whether the button that dismisses the ad is visible before tapping it, because the ad does not always appear and the test must not fail when it is absent. This behavior stays inside `YouTubeScreen`, not `ScreenBase`, because it is specific to this screen and has no reason to be shared with the rest of the framework.

Why? does `pressOkOnKeyboard()` tap a specific point on the screen instead of simulating an "Enter" key press? On a native screen, MobileWright does not offer an equivalent to `press('Enter')` — that mechanism is reserved for web views (*WebView*). On a native interface, confirming the keyboard therefore relies on a real gesture: tapping the visible keyboard button (here, `OK`), exactly as a user would. The method's name faithfully reflects this technical choice.

8 ScreenManager: the conductor, explained line by line

Everything built so far — `ScreenBase`, the *locators*, and the *Screen* classes — now exists independently, but none of these pieces yet knows how to work together. One thing is still missing: a single entry point that the test scenario can call to reach any screen. That is precisely the role of the file `src/Screens/screenManager.ts`, the component with the richest set of design decisions in the entire framework. We explain it here in full, in the exact order it is written.

8.1 The imports: three lines, three different natures

```
import { HomeScreen } from './Home/home.screen';
import { YouTubeScreen } from './YouTube/youtube.screen';
import type { Screen } from '@mobilewright/core';
```

VALUE IMPORT OR TYPE IMPORT

The first two lines import **actual classes**. A class is both a type (it defines a shape) and a value (it exists at runtime, which is why you can write `new HomeScreen(...)`). That is why they are imported without the `type` keyword: the compiled code actually needs the class itself in order to instantiate it further down.

The third line, `import type { Screen }`, imports only a type contract provided by the core of MobileWright (`@mobilewright/core`). The `Screen` type is never instantiated directly in this file — the statement `new Screen()` never appears — it is used solely to tell the compiler that the received parameter matches this shape. Since this type disappears entirely at compile time, writing it with `import type` avoids needlessly bloating the final code.

8.2 Declaring the class and exporting it

```
export class ScreenManager {
  ...
}
```

Why? export precisely this class? Because `ScreenManager` is the only entry point that files outside the framework, such as `fixtures.ts`, need to import. `HomeScreen` and `YouTubeScreen` are also exported from their own files, but in practice no test scenario ever needs to import them directly: they are always reached through the manager. The `export` keyword is the mechanism that makes a class visible outside its own file; without it, `ScreenManager` would stay locked inside `screenManager.ts` and be completely unusable anywhere else.

8.3 The class fields: visibility explained through a real contrast

```
private readonly screen: Screen;
public readonly homeScreen: HomeScreen;
public readonly youTubeScreen: YouTubeScreen;
```

ENCAPSULATION

These three fields are declared here but receive no value yet: their assignment happens in the constructor, right after. This is the ideal moment to answer precisely a central question: why is `screen` `private` while `homeScreen` and `youTubeScreen` are `public`?

`screen` represents the raw connection to the device, as provided by MobileWright. A test scenario never needs to call low-level methods directly on this connection: doing so would bypass all the work

done by HomeScreen and YouTubeScreen, and reintroduce into the tests exactly the technical complexity the framework exists to hide. This field is therefore private: strictly invisible and unusable outside ScreenManager itself. This is where **encapsulation** shows itself: hiding an implementation detail that has no reason to leak to the outside.

homeScreen and youTubeScreen, on the other hand, represent exactly what the test scenario needs to use: the business actions of each screen. Making them public is therefore not incidental, but necessary — without this visibility, no scenario could ever call clickOnHomeButton() or fillSearchField().

readonly applies to all three fields for the same reason explained in the chapter dedicated to ScreenBase: once the manager is built, neither the connection to the device nor the screen instances should ever be replaceable along the way. A scenario should never be able to accidentally write screenManager.homeScreen = somethingElse.

8.4 The constructor: the moment three objects come to life

```
constructor(screen: Screen) {
  this.screen = screen;
  this.homeScreen = new HomeScreen(this.screen);
  this.youTubeScreen = new YouTubeScreen(this.screen);
}
```

DEPENDENCY INJECTION

Why? receive screen as a parameter rather than creating it inside the constructor? Because the connection to a mobile device is provided by the MobileWright engine at the moment a test starts. ScreenManager neither needs to know, nor has the right to decide, how that connection is established — which device, which emulator, which configuration. It simply *receives* an already-initialized connection and passes it along. This is the **dependency injection** principle: a class receives what it depends on instead of manufacturing it itself, which makes it simpler, more predictable, and easier to test in isolation.

The first line, this.screen = screen, creates no new object — there is no new — it simply copies the received reference into the private field. The next two lines are different: new HomeScreen(this.screen) and new YouTubeScreen(this.screen) actually build, for the first time in the program's execution, the two objects that will carry all of the screens' business logic. These are the only two places in the entire application where these classes are instantiated: ScreenManager is the single point of creation, which guarantees that two concurrent, inconsistent instances of HomeScreen can never exist.

Why? pass this.screen — and not a brand-new screen object — to each screen? Because HomeScreen and YouTubeScreen must drive the *same* device as the one received by the manager. This is **composition** at work: ScreenManager does not become a HomeScreen, it owns one, built from the very same dependency it received itself.

8.5 The accessor methods: why return rather than recreate

```
onHomeScreen() {
  return this.homeScreen;
}

onYouTubeScreen() {
  return this.youTubeScreen;
}
```

ABSTRACTION

These two methods do nothing more than return an already-existing reference — no `new` appears here. This is deliberate, and important: the instances of `HomeScreen` and `YouTubeScreen` were already created once, in the constructor, at the previous step. `onHomeScreen()` and `onYouTubeScreen()` are only *access points* to these already-built objects, never factories for new instances. If they recreated a new object on every call, two successive calls to `onHomeScreen()` within the same test could return two different objects, which would break any state consistency across the scenario's steps.

This is also where **abstraction** appears: the test scenario that writes `screenManager.onHomeScreen().clickOnHomeButton()` does not need to know anything about how `HomeScreen` was built, nor about the existence of the private `screen` field. It simply calls a plain method, and everything else — orchestration, dependency injection, field visibility — stays hidden behind this facade.

Why? are these methods called `onHomeScreen()` and `onYouTubeScreen()`, starting with a lowercase letter? TypeScript convention reserves an initial capital letter for class and type names; a method always starts with a lowercase letter. Following this rule everywhere avoids any visual ambiguity between "this is a class" and "this is an action to call".

9 Fixtures: wiring the framework into the test runner

The file `src/fixtures.ts` is the bridge between `ScreenManager` — the framework we just built — and the scenario files in the `tests/` folder. We explain it below in the order it is written.

```
import { test as base } from '@mobilewright/test';
import { ScreenManager } from './Screens/screenManager';

type Fixtures = {
  screenManager: ScreenManager;
};

export const test = base.extend<Fixtures>({
  screenManager: async ({ screen }, use) => {
    const screenManager = new ScreenManager(screen);
    await use(screenManager);
  },
});

export { expect } from '@mobilewright/test';
```

9.1 The `Fixtures` type: describing a resource before providing it

```
type Fixtures = {
  screenManager: ScreenManager;
};
```

TYPE CONTRACT

This type creates nothing: it describes, for the compiler, the exact shape of the new resource this file is about to add to the test system — a property named `screenManager`, of type `ScreenManager`. Thanks to this declaration, every test that writes `async ({ screenManager })=> {...}` benefits from auto-completion and type checking on `screenManager`, exactly like on any other variable.

9.2 Extending the base test system

FIXTURE EXTENSION

The call `base.extend<Fixtures>(...)` extends the test system provided by MobileWright to add a new resource named `screenManager`, automatically available in every test. The `Fixtures` type declared just above serves only to describe, for the compiler, the shape of this new resource.

Why? go through `base.extend(...)` rather than creating a `ScreenManager` directly in every scenario file? Because that would force every scenario to know the manager's construction details — exactly the kind of detail this framework is designed to hide. By centralizing this construction inside `fixtures.ts`, every scenario in the project gets a ready-to-use `screenManager`, built the same way, without duplicating this code.

9.3 What the provided function actually does

ONE CREATION PER TEST

The provided function already receives `screen` — the connection to the device, prepared by MobileWright before this code even runs —, creates *exactly once per test* a `new ScreenManager(screen)`, then hands it over via `use(screenManager)`.

9.4 The precise role of `use(...)`

HANDOFF POINT

Why? call `use(screenManager)` rather than simply doing `return screenManager`? `use(...)` is not a classic return: it is the exact point where the MobileWright test runner takes back control, runs the body of the test while supplying it with the built `screenManager`, then comes back *after* this call once the test has finished. This is what would allow, if needed, cleanup code to be added right after the line `await use(screenManager)` — for example closing a resource — without ever touching the scenario files. In this project no cleanup is needed, but the structure of the mechanism stays the same.

Why? is it precisely here, and nowhere else, that the `screen` connection meets `ScreenManager`? Because it is `fixtures.ts`'s job to make that link, once, in the same place, for every scenario in the project. The manager itself, as explained in the previous chapter, does not know where `screen` comes from: that is simply not its job to know.

10 The scenario: concrete proof of the benefit

All this construction work only makes sense if it results in a test scenario that is simple to read, even for someone who does not code. Here is the file `tests/youTubeScenario.test.ts` exactly as it is written in this project.

```
import { test, expect } from '../src/fixtures';

test('YouTube scenario', async ({ screenManager }) => {
  await screenManager.onHomeScreen().clickOnHomeButton();
  await screenManager.onHomeScreen().middleSwipeUp();
  await screenManager.onHomeScreen().clickOnYouTubeApplicationIcon();
  await screenManager.onHomeScreen().bottomSwipeUp();
  await screenManager.onHomeScreen().middleSwipeUp();
  await screenManager.onHomeScreen().clickOnYouTubeApplicationIcon();
  await screenManager.onYouTubeScreen().clickOnSearchIcon();
  await screenManager.onYouTubeScreen().fillSearchField('MobileWright');
  await screenManager.onYouTubeScreen().pressOkOnKeyboard();
  await screenManager.onYouTubeScreen().clickOnVideo();
  await screenManager.onYouTubeScreen().clickOnSkipAdButton();
```

```
  await screenManager.onYouTubeScreen().swipeVideoProgressSliderRight(1000);
});
```

10.1 What this file never does

NOTHING TO BUILD HERE

The signature `async ({ screenManager })=>` directly receives the object prepared by `fixtures.ts`: no line in this file creates a connection, configures a device, or builds an object — all of that has already been done upstream.

10.2 What this file tells

THE SAME PATTERN, REPEATED

The test simply tells, in order, the user's story: go back to the home screen, scroll, open YouTube, search for a video, open it, dismiss the ad if it appears, then move forward in playback. Every line follows exactly the same pattern explained in detail in the previous chapter: `screenManager` gives access to the screen (`onHomeScreen()` or `onYouTubeScreen()`), which itself gives access to a business action already named in an understandable way.

10.3 The proof of maintainability

Why? will this file never need to change, even if the application changes? If tomorrow the search field's identifier changes in the YouTube application, exactly one line will need to be updated — inside `youtube.locator.ts` — and this scenario file will stay untouched. This is precisely the benefit this document set out to demonstrate from its very first chapter.

11 In summary

The framework built in this document rests on a simple idea behind every technical decision: each responsibility has exactly one place to live. Targeting an element lives in a *locator*. Acting on a screen lives in a *Screen* class. Gestures shared across screens live in `ScreenBase`. Choosing the active screen lives in `ScreenManager`. Wiring into the test engine lives in `fixtures.ts`. None of this is arbitrary: every visibility level — `private`, `protected`, or `public` —, every `export`, every instantiation with `new`, answers a precise question about who needs to know what. It is this discipline that lets the final scenario stay readable as a single sentence.